API in cybersecurity

Introduction-

In cybersecurity, an API (Application Programming Interface) is a set of rules that allows different software applications to communicate with each other.

Example

When we use a mobile app to check the weather:

The app sends a request to a weather server through an API.

The server sends back weather data.

The app displays the information to you.

The API acts like a messenger between the app and the server.

Matter in Cybersecurity as APIs often handle sensitive data such as:

- Usernames and passwords
- Personal information
- Payment details
- Company data

If an API is not secured properly, attackers may:

- Access confidential data
- Steal user accounts

- Modify information
- Launch attacks on systems

For analysing work of API and its importance in cybersecurity we select following API-

API : JSONPlaceholder API Documentation

1. Executive Summary

JSONPlaceholder is a public fake-data REST API designed for development and testing. It intentionally prioritizes ease of use over security controls because it does not contain real customer data.

The review found:

Finding	Severity
No authentication required	Medium
No authorization controls	Medium
No rate limiting	Medium
CORS allows all origins	Low
Excessive data exposure risk if used as a production model	Low
Limited error information exposure	Low

Because the API contains only test data, these issues do not represent an immediate business threat. However, if a similar design were deployed in production, several findings would become High or Critical severity.

2. Documentation Review

Base URL

https://api.jsonplaceholder.dev

Resources

- /users
- /posts
- /comments
- /albums
- /photos
- /todos

Supports:

- GET
- POST
- PUT
- PATCH
- DELETE

All responses are JSON.

---

3. Endpoint Testing Results

Test 1 – Get User Information

Request

GET /users/1

Headers

Accept: application/json

Authentication

None required.

Example Response


```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "..."
}
```


Observation

User information is publicly accessible.

Security Risk

No access controls exist.

Severity

Medium

Business Explanation

Anyone can retrieve user records without proving their identity. In a real customer-facing system this could expose personal information.

Recommendation

- Require authentication.
- Apply role-based access controls.
- Restrict sensitive user data.

Test 2 – Retrieve Posts

Request

GET /posts

Authentication

None required.

Observation

Entire dataset can be enumerated.

Security Risk

Mass data harvesting would be possible if real information were present.

Severity

Medium

Recommendation

- Require API authentication.
- Apply pagination.
- Monitor bulk requests.

Test 3 – Create Resource

Request

POST /posts

Example body:

```
{
  "title":"Test",
  "body":"Security Review",
  "userId":1
}
```

Observation

Documentation indicates POST requests are accepted and return success responses.

Security Risk

Without authentication, an attacker could submit arbitrary content in a production environment.

Severity

High (if production)
Low (for this demo API)

Recommendation

- Require authenticated users.

- Validate input.
- Log creation events.

Test 4 – Error Handling Review

Example Error

```
{
  "error":"Not Found"
}
```

or

```
{
  "error":"Post not found"
}
```

Observed documentation shows relatively generic error messages.

Security Risk

Minimal information disclosure.

Severity

Low

Recommendation

Continue avoiding stack traces and internal system details.

---

4. Authentication Assessment

Findings

Documentation explicitly states that authentication is not required.

Risk

- Anonymous access
- No accountability
- No user verification

Severity

Medium

Business Impact

An organization would be unable to determine who accessed data or performed actions.

Recommended Controls

- OAuth 2.0
- API keys
- JWT-based authentication
- Audit logging

---

5. Authorization Assessment

Findings

No role-based access control exists.

Any user can access the same resources.

Risk

Broken Access Control if copied into production.

Severity

High (production scenario)

Business Impact

Customers could potentially access other customers' information.

Remediation

- Enforce authorization checks per request.
- Implement least-privilege permissions.
- Validate ownership of resources.

---

6. Header Review

Observed/Documented Headers

Content-Type: application/json
Access-Control-Allow-Origin: *

CORS is enabled for all origins.

Risk

Any website can make requests directly to the API.

Severity

Low

Business Impact

In production this may increase abuse opportunities.

Remediation

Restrict CORS to approved domains.

Example:

Access-Control-Allow-Origin: https://company.com

---

6. Rate Limiting Review

Documentation states there are no rate limits.

Risk

Potential for:

- Automated scraping
- Excessive traffic
- Resource abuse

Severity

Medium

Business Impact

Unexpected infrastructure costs and degraded service availability.

Remediation

Implement:

- Per-user limits
- Per-IP limits

- Abuse monitoring
- Traffic anomaly detection

---

7. Data Exposure Review

Current data includes:

- Names
- Usernames
- Email fields
- Posts
- Comments

Since the dataset is fake, exposure risk is low. However, the API demonstrates how real user data could be exposed if proper controls were absent.

Severity

Low

Remediation

- Return only required fields.
- Mask sensitive data.
- Apply data classification policies.

---

9. Overall Risk Matrix

Category	Severity

Authentication	Medium
Authorization	Medium
Rate Limiting	Medium
CORS Configuration	Low
Data Exposure	Low
Error Handling	Low

---

8. Final Conclusion

JSONPlaceholder is intentionally designed as an open testing API and is functioning as expected for that purpose. The most significant observations are the absence of authentication, authorization, and rate limiting controls. These are acceptable for a public demo environment but would represent serious security concerns in a production API.

Priority Remediation Steps for Production APIs

1. Implement strong authentication (OAuth2/JWT).
2. Enforce object-level authorization.
3. Add rate limiting and abuse protection.
4. Restrict CORS policies.
5. Reduce unnecessary data exposure.
6. Enable security logging and monitoring.
7. Conduct regular API security reviews aligned with the OWASP API Security Top 10 framework.